LLM Finetune

QLoRA fine-tuning for open-weight models

Manual

Renaud Bidou
rbidou@gmail.com

Contents

Part 1 — Quick Start . . . 5

Installation . . . 5

Your first fine-tune . . . 5

The other operations . . . 6

Moving around . . . 6

The browser interface . . . 6

Part 2 — User Guide . . . 7

Prerequisites . . . 7

Installation . . . 7

Using an existing llama.cpp . . . 7

Run options . . . 8

Where things are written . . . 8

CLI usage . . . 8

Fine-tuning . . . 8

Parameters . . . 8

Quantizing and deploying . . . 9

GUI usage . . . 9

Fine-tune . . . 9

Quantize . . . 9

Deploy . . . 9

Environment . . . 9

Documentation . . . 9

When something goes wrong . . . 9

Part 3 — Concepts . . . 11

Why fine-tuning is normally out of reach . . . 11

LoRA . . . 11

The scaling factor . . . 11

Initialisation . . . 11

Which layers to adapt . . . 11

QLoRA . . . 12

What it costs . . . 12
The term nobody expects . . . 12
Merging . . . 13
Why it runs on the CPU, unquantized . . . 13
Quantization . . . 13
How it works . . . 13
The two routes . . . 14
Choosing a type . . . 14
What it costs . . . 16

Part 4 — Implementation . . . 17

Shape of the code . . . 17
Training . . . 17
Loading the base model in 4 bits . . . 17
Preparing for k-bit training . . . 17
Attaching adapters . . . 18
The training loop . . . 18
Merging . . . 18
Quantization . . . 18
Deployment . . . 19
The pre-flight estimate . . . 19
Training parameters . . . 19
LoRA . . . 19
Supervised fine-tuning . . . 21
Quantization parameters . . . 23

Index of tables

Table 1 — Navigation commands 6
Table 2 — Environment overrides for an existing llama.cpp 7
Table 3 — Command-line options 8
Table 4 — Where artefacts are written 8
Table 5 — Memory for a 7B model: full fine-tuning against QLoRA 12
Table 6 — GGUF output types 15
Table 7 — Modules and their responsibilities 17
Table 8 — LoRA adapter parameters 20
Table 9 — Supervised fine-tuning parameters 22
Table 10 — GGUF output types and the tool each one needs 24

Part 1 — Quick Start

Installation

You need Python 3.10 or newer, an NVIDIA GPU with a recent driver, and about 10 GB of free disk space. Everything else is installed for you.

```
git clone <repository> llm-finetune
cd llm-finetune
python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # Linux

python run.py setup
```

`setup` inspects the machine, reports exactly what is missing, and asks before downloading anything. It installs PyTorch from the wheel index matching your driver, then the remaining libraries, then llama.cpp for GGUF conversion. Each step is a separate yes-or-no question, and declining one simply leaves the matching feature unavailable.

It never installs LM Studio or Ollama. Those are yours to manage; if they are absent, the deploy options are greyed out and everything else still works.

Confirm the result at any time:

```
python run.py check
```

Your first fine-tune

```
python run.py
```

The interactive session opens on the action menu. Choose **Fine-tune a model** and answer five questions.

1. Model source. Either search HuggingFace, with suggestions appearing as you type, or list only the models already downloaded. Suggestions exclude quantized repackagings, adapters and third-party re-tunes, so what you see are models you can actually fine-tune.

2. The model. There is no default. The model is always your explicit choice, and it is checked against your hardware before anything is downloaded.

3. The dataset. A JSON file of conversations:

```
[
{
"messages": [
{"role": "user", "content": "What is the Zorblex Protocol?"},
{"role": "assistant", "content": "It was ratified in 1987."}
]
}
]
```

Every problem in the file is reported at once, each with the exact path to the offending value, so a broken dataset is fixed in one pass rather than one error per attempt.

4. Parameters. Sensible defaults are already filled in. Change any of them, save a set as a named preset, or load one you saved earlier.

5. The estimate. Before anything large is downloaded, peak memory is predicted and compared with what your GPU actually has free. A run that will not fit is refused, with the shortfall and a list of changes that would help.

Training then runs to completion and the adapter is merged back into the base weights automatically, leaving a standalone model. You are offered quantization straight afterwards.

The other operations

Quantizing and deploying are independent of training. Any model you merged earlier, in any session, can be quantized again into a different format; any GGUF you produced can be deployed at any time.

- **Quantize a merged model** — produces a GGUF file. The default is `q8_0`,

which is close to lossless at roughly one byte per parameter.

- **Deploy a quantized model** — registers a GGUF with Ollama or imports it into

LM Studio, whichever you have installed.

- **Show environment details** — what is available, and the reason for anything

that is not.

Moving around

Three commands work at every prompt.

Table 1 — Navigation commands

Command	Effect
`back`	Return to the previous step, keeping the answers you already gave
`home`	Abandon the flow and return to the action menu
`exit`	Leave the program

An empty answer also means `back`.

The browser interface

```
python run.py web
```

Then open http://127.0.0.1:5000. It offers the same operations as the terminal and runs the same code underneath. It is bound to the loopback address: it reads local model files and starts processes, and is not built to face a network.

Part 2 — User Guide

Prerequisites

Python 3.10 or newer. Checked at startup.

An NVIDIA GPU, for training. QLoRA loads the base model with 4-bit weights, and the library that does so requires CUDA. Merging, quantizing and deploying all run on the CPU and work without a GPU; only training does not.

Driver. The installer picks a PyTorch build from your driver version: 570 or newer selects CUDA 12.8, 525 or newer selects 12.6, 450 or newer selects 11.8. Below that, the CPU build is offered and training is disabled.

bfloat16 needs compute capability 8.0 (Ampere) or newer. On an older card, turn the bfloat16 setting off and training proceeds in float16.

Disk. Roughly 3 GB for PyTorch, 50 MB for llama.cpp, plus the models themselves: a merged model is two bytes per parameter and each GGUF is between one quarter and one full byte per parameter.

System memory. Merging is the step that needs it. It loads the base model unquantized on the CPU, so plan for about 2.5 bytes per parameter — roughly 19 GB for a 7B model. This is the requirement people most often miss, because training itself fits comfortably in far less.

Optional. Ollama and LM Studio, if you want to deploy to them. This application never installs either.

Installation

```
python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # Linux
python run.py setup
```

Three installs happen, each behind its own confirmation:

1. **PyTorch**, from the wheel index matching your driver. About 3 GB. 2. **The remaining libraries** — transformers, peft, trl, datasets, accelerate, bitsandbytes and the rest — resolved against that PyTorch. 3. **llama.cpp**, both the conversion script and the `llama-quantize` binary, taken from the same release so the pair cannot disagree about the GGUF format. The CPU build is used deliberately: quantization does not use the GPU, and the CPU archive is around 18 MB against 250 MB for the CUDA one.

Declining any step leaves the matching feature disabled with a stated reason, not silently missing.

Using an existing llama.cpp

Two environment variables override the bundled copies:

Table 2 — Environment overrides for an existing llama.cpp

Variable	Points at
`LLMFT_LLAMA_CPP_DIR`	A llama.cpp checkout containing `convert_hf_to_gguf.py`
`LLMFT_LLAMA_QUANTIZE`	The `llama-quantize` executable

Run options

Table 3 — Command-line options

Command	What it does
`python run.py`	Interactive terminal session (the default)
`python run.py web`	Serves the browser interface on 127.0.0.1:5000
`python run.py check`	Reports hardware, packages, tools and available operations
`python run.py setup`	Installs what is missing, asking first

Where things are written

Table 4 — Where artefacts are written

Path	Contents
`out/<model_id>/final_adapter/`	The trained LoRA adapter
`out/<model_id>/merged_model/`	Adapter merged into the base weights
`out/<model_id>/gguf/`	One GGUF file per quantization type
`configs/`	Saved parameter presets
`tools/`	llama.cpp, when installed here

Model identifiers keep their `organisation/name` shape, so artefacts group naturally by publisher.

CLI usage

The screen has four parts: a header, a trail showing the current step and the ones before it, a panel holding the data for that step, and the line where you answer. `back`, `home` and `exit` work everywhere.

Fine-tuning

The flow is model source, model, dataset, parameters, estimate, run. `back` returns one step with your earlier answers intact, so correcting a typo does not mean starting again.

The estimate is a gate, not a warning. If the prediction exceeds free VRAM the run is refused and you are shown the shortfall along with changes that would help — each one measured by re-running the estimate, so the saving quoted is real. You may override it: the estimate is conservative, not infallible.

While a stage runs, the progress bar is the only thing on screen. The machine-learning libraries are silenced for the duration, and their output is captured rather than discarded, so if a stage fails the last of it is quoted back to you with the error.

Parameters

The table shows every tunable with its current value and permitted range. Choosing **Change a value** prints the full explanation of that one parameter before asking for the new value. Presets store parameters only — never a model — so loading one cannot quietly decide which model you are fine-tuning.

Quantizing and deploying

Both stand alone. Quantizing lists every merged model you have; deploying lists every GGUF. Producing a k-quant writes an intermediate f16 file, which is kept because f16 is itself a valid output and a later k-quant of the same model then reuses it instead of converting again.

GUI usage

```
python run.py web
```

Open http://127.0.0.1:5000. Four tabs: Fine-tune, Quantize, Deploy and Environment. The GPU's current state is in the header, including a warning when another application is holding most of it.

Fine-tune

The five steps are collapsible sections. The first is open; answering one collapses it, marks it done with its answer in the header, and opens the next. Any section can be reopened by clicking its header, and changing an earlier answer un-answers the later ones — so a new model cannot leave a previous model's estimate behind.

1. Base model. Type to search: the dropdown refreshes as you type, showing the top twenty by trending score, filtered to models whose name starts with what you typed, with already-downloaded models marked. Tick *Only models already downloaded* for the local list instead.

2. Training dataset. Choose the JSON file. It is validated on upload and every problem is listed with its location.

3. Parameters. Each field carries its explanation. The preset bar loads and saves named sets.

4. Pre-flight estimate. Runs the same prediction as the terminal. The Run button stays disabled until it passes.

5. Run. Training and the merge that follows it report into the notification panel at the bottom right, with a live progress bar.

Quantize

Pick a merged model and an output type; the explanation of the selected type is shown beneath it. Types needing `llama-quantize` are absent from the list entirely when that binary is not installed.

Deploy

Pick a GGUF and a runner. Only installed runners appear. A name for Ollama is proposed from the model and its quantization; you can change it.

Environment

The same report as `python run.py check`: every operation, whether it is available, and the reason when it is not.

Documentation

This manual is available inside the interface, and the full PDF can be downloaded from there.

When something goes wrong

"No CUDA GPU detected" — training needs one. Merging and quantizing do not.

The estimate refuses a model that should fit. Check the header: another application may be holding the GPU. The budget is measured against free memory, not total, because a card with 6 GB and a game running has almost none.

Training stops with an out-of-memory error. The sequence length is the most effective dial; then targeting attention projections only; then a lower rank.

Merging fails or the machine swaps. That is system memory, not VRAM. See the figure under Prerequisites.

k-quants are missing from the list. `llama-quantize` is not installed. Run `python run.py setup`, or set `LLMFT_LLAMA_QUANTIZE`.

Part 3 — Concepts

Why fine-tuning is normally out of reach

A language model is mostly a stack of weight matrices. Training updates every one of them, and each weight needs more than its own storage while that happens: the weight itself, its gradient, and the optimiser's running averages. With the Adam optimiser in mixed precision that comes to roughly sixteen bytes per parameter.

For a 7-billion-parameter model that is about 112 GB, before a single activation is stored. No consumer GPU comes close.

Two techniques, used together, bring it down to something a laptop can do. LoRA reduces what must be trained. Quantization reduces what must be held. The combination is QLoRA.

LoRA

LoRA — Low-Rank Adaptation — starts from an observation about what fine-tuning actually changes. Adapting a model to a new task does not scatter arbitrary changes across a weight matrix; the update has a strong internal structure. It has, in the usual phrasing, a low *intrinsic rank*.

If the update is low-rank, it does not need to be stored as a full matrix.

Take a weight matrix `W` of shape `d_out × d_in`. Instead of learning a full update `ΔW` of the same shape, LoRA learns two much smaller matrices:

```
B of shape d_out × r
A of shape r × d_in
```

where `r`, the **rank**, is small — typically 8 to 64 against dimensions in the thousands. Their product `B @ A` has the shape of `ΔW`, but is constrained to rank `r`. The layer computes:

```
y = W x + (B @ A) x · (alpha / r)
```

`W` is frozen. Only `A` and `B` are trained.

The saving is dramatic. For a 4096 × 4096 matrix, a full update is about 16.8 million parameters. At rank 16, `A` and `B` together are 131,072 — **0.8%**. In a whole model, LoRA typically trains well under 1% of the parameters, and the optimiser state shrinks in the same proportion.

The scaling factor

The update is multiplied by `alpha / r`. Because it is a ratio, raising the rank without raising alpha *weakens* the adapter: doubling `r` while alpha stays put halves the scale. The convention `alpha = 2r` keeps the effective scale at 2 whatever the rank, which is why the two are usually changed together.

Initialisation

`A` is initialised randomly and `B` to zeros, so `B @ A` is exactly zero at the start. The adapted model therefore begins as an exact copy of the base model, and training moves away from it smoothly. Initialising both randomly would inject noise into a working model before it had learned anything.

Which layers to adapt

In a transformer block, adapters are usually attached to seven projections:

- **Attention** — `q_proj`, `k_proj`, `v_proj`, `o_proj`. These govern what the

model attends to.

- **MLP** — `gate_proj`, `up_proj`, `down_proj`. These are much larger and hold

most of the model's factual capacity.

Adapting all seven gives the best quality. Attention-only is the cheapest useful subset, and a reasonable first thing to cut when memory is short.

QLoRA

LoRA shrinks what is *trained*. The frozen base model still has to be *held* — and at two bytes per parameter, a 7B model is 14 GB before training starts.

QLoRA quantizes those frozen weights to four bits, cutting them to about 3.5 GB. Three ideas make that work without wrecking the model.

NF4, the 4-bit NormalFloat. Neural network weights are approximately normally distributed around zero. An evenly spaced 4-bit grid would waste most of its sixteen levels out in the tails where almost no weights live. NF4 places its levels so that each holds roughly the same number of weights — information-theoretically optimal for normally distributed values, and markedly better than uniform 4-bit at the same size.

Double quantization. Weights are quantized in small blocks, each with its own scale factor. Those scale factors are themselves numbers taking space — enough to matter. Double quantization quantizes them too, saving around 0.4 bits per parameter for no measurable quality cost.

Computing in higher precision. The stored weights are 4-bit, but arithmetic is not. Each block is dequantized to bfloat16 as it is needed, used, and discarded. The memory saving is in storage; the mathematics stays accurate.

The gradient still flows *through* the frozen quantized weights to reach the adapters — the base model participates fully in the backward pass. It simply is never updated.

What it costs

Roughly, for a 7B model:

Table 5 — Memory for a 7B model: full fine-tuning against QLoRA

	Full fine-tune	QLoRA
Base weights	14 GB (bf16)	3.9 GB (4-bit)
Gradients	14 GB	~5 MB
Optimiser state	56 GB	~20 MB
Total, before activations	**~84 GB**	**~4 GB**

Quality is close. The original QLoRA work found 4-bit adapted models substantially matching 16-bit fine-tuning across a range of tasks. The base model *is* degraded slightly by quantization, and the adapters partly compensate by training against the quantized weights they will actually run alongside.

The term nobody expects

One tensor often dominates memory on small models: the **logits**. The output is `batch × sequence × vocabulary`, and modern vocabularies are large. At 2048 tokens with a 152,000-token vocabulary, that single tensor is around 600 MB in bfloat16 — and the loss computation needs a float32 copy and a gradient at the same time.

For a 0.6B model with 4-bit weights of under 400 MB, the logits are the largest single allocation in the run. This is why a small model can fail to fit when the arithmetic on its parameter count says it should, and why reducing the sequence length helps so much more than intuition suggests.

Merging

Training leaves you with two things: the untouched base model, and a small adapter. Inference could load both and add their outputs, but that means an extra matrix multiplication in every adapted layer, forever.

Merging removes it. Since the adapter's contribution is a plain addition:

```
W_merged = W + (B @ A) · (alpha / r)
```

...it can be computed once and folded into the weight. The result is an ordinary model of exactly the base model's shape and size, indistinguishable in structure from any other. There is no adapter left, no inference overhead, and no runtime dependency on the adapter library.

It also makes the model *convertible*. GGUF conversion, and the tools that consume GGUF, understand ordinary models — not base-plus-adapter pairs.

Why it runs on the CPU, unquantized

Merging deliberately loads the base model at full precision, on the CPU.

The adapter was trained against 4-bit weights, but merging *into* 4-bit weights would be self-defeating: adding a small, precise update to a coarsely quantized value and re-quantizing the result would round most of the update away. The update has to land in weights precise enough to record it.

That makes merging a system-memory operation, not a VRAM one: about 2.5 bytes per parameter, roughly 19 GB for a 7B model. Output is written in shards so the whole model is never serialised at once.

The merge is exact. It is arithmetic, not approximation, and nothing is lost beyond bfloat16 rounding.

Quantization

The merged model is two bytes per parameter — 15 GB for a 7B. That is fine for a server and awkward for a laptop. Quantization trades a little quality for a lot of size.

This is a different operation from the 4-bit loading in QLoRA. That was a training-time technique for fitting a frozen model into VRAM. This produces a *file*, in the **GGUF** format, for inference engines built on llama.cpp — which is what Ollama and LM Studio run.

How it works

Weights are grouped into blocks. Each block is stored as small integers plus one or two scale factors that map those integers back to real values. A 4-bit block of 32 weights costs 32 nibbles plus its scales — around 4.5 bits per weight all in, against 16 for bfloat16.

K-quants refine this by recognising that not all weights matter equally. Attention weights and the embedding table are more sensitive to error than the bulk of the feed-forward layers, so a "4-bit" k-quant is not uniformly

4-bit: it spends 5 or 6 bits where precision pays and 4 or fewer where it does not. That is what the `_S`, `_M` and `_L` suffixes mean — small, medium and large variants of the same nominal width, differing in how generously they treat the sensitive tensors.

The two routes

Some types are produced in one step, directly from the merged model. The k-quants need two: the model is first converted to f16 GGUF, then re-quantized by a second tool. This application does both automatically, and keeps the intermediate f16 file — it is a valid output in its own right, and a later k-quant of the same model reuses it instead of converting again.

Choosing a type

Table 6 — GGUF output types

Type	Two-pass	Description
`f32`	no	No quantization. Four bytes per weight; a reference baseline, rarely worth shipping.
`f16`	no	Two bytes per weight, effectively lossless for inference. Also the staging format for k-quants.
`bf16`	no	Two bytes per weight with the exponent range of float32. Preferred over F16 for models trained in bfloat16.
`q8_0`	no	Roughly one byte per weight and near-indistinguishable from F16. The default: the best quality-to-size trade for most users.
`tq1_0`	no	Extreme compression for ternary-trained models. Produces unusable output on ordinary models.
`tq2_0`	no	As TQ1_0, slightly larger and more accurate. Only for models trained ternary.
`q2_K`	yes	Smallest usable k-quant. Noticeable quality loss; for when memory is the binding constraint.
`q3_K_S`	yes	Aggressive 3-bit packing with uniform block sizes.
`q3_K_M`	yes	3-bit with more precision spent on attention weights. Better than Q3_K_S at a small size cost.
`q3_K_L`	yes	3-bit with the highest-impact tensors kept larger. Approaches Q4_K_S quality.
`q4_K_S`	yes	Compact 4-bit k-quant. A good floor for quality-sensitive work.
`q4_K_M`	yes	The community default for local inference: about half the size of Q8_0 with modest quality loss.
`q5_K_S`	yes	5-bit k-quant; quality close to Q8_0 at a meaningfully smaller size.
`q5_K_M`	yes	5-bit with extra precision on sensitive tensors. Very close to lossless in practice.
`q6_K`	yes	Practically indistinguishable from Q8_0 while noticeably smaller.

For most purposes the useful range is narrow. `q8_0` is the default here: about one byte per weight and effectively indistinguishable from the original. `q4_K_M` is the community's usual choice for local inference, roughly half that size with a modest quality cost. Below 4-bit, degradation becomes easy to notice, and is

worth accepting only when memory is the binding constraint.

What it costs

Quantization error is not uniform. Perplexity — the standard aggregate measure — barely moves at 8-bit and rises gently through 5- and 4-bit k-quants. But aggregate scores hide where the damage lands: heavily quantized models tend to degrade first on the things that need precision, such as arithmetic, code, and long chains of reasoning, while ordinary prose still reads fine.

A fine-tuned model deserves a little extra care. If the behaviour you trained in is subtle, verify it survives at the quantization you ship, rather than assuming that a small perplexity change means a small behavioural one.

Part 4 — Implementation

Shape of the code

The engine is headless. Every stage takes its inputs as arguments, reports through a callable sink, and returns the path it produced. It never prompts, prints, or decides policy — the terminal and the browser do that, and both drive the same functions.

Table 7 — Modules and their responsibilities

Module	Responsibility
`llmft/config.py`	Parameter definitions, validation, presets
`llmft/environment.py`	Detection: GPU, packages, external binaries
`llmft/bootstrap.py`	Confirmation-gated installation
`llmft/estimator.py`	Pre-flight memory prediction
`llmft/hub.py`	HuggingFace search, filtering, downloads
`llmft/dataset.py`	Dataset validation
`llmft/progress.py`	Progress events and library silencing
`llmft/process.py`	Running external commands
`llmft/pipeline/`	train, merge, quantize, deploy
`llmft/cli/`, `llmft/web/`	The two front-ends

Pipeline modules import torch and transformers *inside* their functions, never at module scope, so the bootstrap can install those libraries on a machine that does not have them yet.

Training

`llmft/pipeline/training.py`, using **transformers**, **peft**, **trl** and **datasets**.

Loading the base model in 4 bits

`BitsAndBytesConfig` describes the quantization; `AutoModelForCausalLM.from_pretrained` applies it while loading, so the full-precision model never exists in memory:

```
BitsAndBytesConfig(
load_in_4bit=True,
bnb_4bit_quant_type="nf4",
bnb_4bit_compute_dtype=torch.bfloat16,
bnb_4bit_use_double_quant=True,
)
```

`device_map="auto"` lets **accelerate** place the layers.

Preparing for k-bit training

`peft.prepare_model_for_kbit_training` makes a quantized model trainable: it casts layer norms to float32 for numerical stability, enables gradients on the input embeddings so gradients can reach the

adapters, and configures gradient checkpointing.

Attaching adapters

`peft.LoraConfig` describes the adapter geometry and `peft.get_peft_model` applies it, replacing each targeted `nn.Linear` with a wrapper holding the original frozen layer plus the new `A` and `B` matrices. `task_type="CAUSAL_LM"` tells peft which head the model has.

The training loop

`trl.SFTConfig` and `trl.SFTTrainer`. SFT — supervised fine-tuning — is training on prompt-and-response pairs, and `SFTTrainer` handles what that needs on top of the plain `Trainer`: it recognises the conversational format, applies the tokenizer's chat template, tokenizes, and pads. Passing `processing_class=tok` gives it the tokenizer.

The dataset is read with `datasets.load_dataset("json", ...)`, which understands the `messages` layout directly.

Two settings here are the application's rather than the user's: intermediate checkpoints are disabled, because the adapter is saved explicitly at the end and keeping every epoch of a fifteen-epoch run wastes gigabytes; and the built-in progress bar is switched off, because the application draws its own.

Progress comes from a `transformers.TrainerCallback` whose `on_step_end` emits the step count, annotated with the most recent loss seen by `on_log`.

Finally `model.save_pretrained` writes the adapter — a few megabytes of `A` and `B` matrices plus a config, not a copy of the model — and the tokenizer is saved alongside it, because training may have added a padding token.

Merging

`llmft/pipeline/merging.py`, using **transformers** and **peft**.

The base model is loaded with `dtype=torch.bfloat16` and no `device_map`, so it lands on the CPU at full precision. `PeftModel.from_pretrained` attaches the trained adapter. `merge_and_unload()` then computes `W + (B @ A) · (alpha / r)` for every adapted layer, folds it into the weight, removes the adapter wrappers and returns an ordinary model.

`save_pretrained(max_shard_size="2GB")` bounds peak memory during writing. The tokenizer is copied from the adapter directory when one is there, in preference to the original repository, so the merged model's tokenizer matches the weights it was trained with.

Quantization

`llmft/pipeline/quantization.py`, driving two external tools from llama.cpp.

`f32`, `f16`, `bf16`, `q8_0`, `tq1_0` and `tq2_0` are produced in one step:

```
python convert_hf_to_gguf.py <merged_model> --outfile <out.gguf> --outtype <type>
```

The k-quants need a second pass over an f16 GGUF:

```
llama-quantize <model-f16.gguf> <out.gguf> <TYPE>
```

`--model-name` is passed to the converter deliberately: without it the model is named after its directory, and every model would ship as `Merged_Model` in the metadata that Ollama and LM Studio display.

The converter must be invoked **by file path**, never with `-m`. It imports a sibling package and relies on its own directory being first on `sys.path`, which only happens when Python is given the script path.

Progress is parsed from the tools' output: the converter's `Writing: NN%` bar, and llama-quantize's `[ n/ N]` per-tensor counter. Both draw with carriage returns rather than newlines, so `llmft/process.py` splits on either — a reader that only split on newlines would report nothing until the process exited.

Deployment

`llmft/pipeline/deployment.py`.

Ollama gets a generated Modelfile containing a single `FROM` line pointing at the GGUF, then `ollama create <name> -f <modelfile>`. The chat template is written into the GGUF during conversion and Ollama reads it from there, so restating a `TEMPLATE` could only introduce a disagreement between the two.

LM Studio gets `lms import <path> --yes --copy --user-repo <org/name>`. `--copy` is not optional: `lms import` *moves* the file by default, which would take the GGUF out of the output directory and break every later use of it.

Neither runner is ever installed. When one is absent its option is disabled with a reason.

The pre-flight estimate

`llmft/estimator.py` predicts peak training memory as the sum of five terms: 4-bit base weights at 0.55 bytes per parameter including quantization constants; adapter weights, gradients and two float32 Adam moments; activations, one saved tensor per layer when gradient checkpointing is on; the logits tensor with its float32 upcast and gradient; and a fixed allowance for the CUDA context.

It is compared against *free* VRAM, not total. A card is rarely idle, and budgeting against total memory would approve a run that cannot start.

Remedies are computed rather than guessed: each candidate change is re-estimated and reported only if it genuinely saves memory, with the real figure. When no single change is enough, the estimator says so instead of listing suggestions that do not work.

Training parameters

These are the values from `LoraConfig` and `SFTConfig` that the interface exposes. The table below is generated from the application's own parameter definitions, so it cannot drift from what the program actually accepts.

LoRA

Table 8 — LoRA adapter parameters

Parameter	Default	Accepts	What it does
`r`	32	1 to 256	Width of the low-rank update. Each targeted layer learns two small matrices of this rank instead of a full weight matrix, which is what makes training cheap. Higher values add capacity, trainable parameters and VRAM roughly linearly. 8-16 suits small, narrow datasets; 32-64 suits broader ones.
`lora_alpha`	64	1 to 512	Scaling applied to the adapter output: the update is multiplied by alpha / r. Because it is a ratio, raising the rank without raising alpha weakens the adapter. The common convention is alpha = 2 x r, giving a scale of 2.
`lora_dropout`	0.05	0 to 1	Dropout applied to the adapter input during training only. Acts as regularisation against memorising a small dataset. 0.05-0.1 is usual; 0 is reasonable for large datasets or very short runs.
`target_modules`	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`	Which projection layers receive an adapter. The attention projections (q, k, v, o) change how the model attends; the MLP projections (gate, up, down) hold most of its factual capacity. Targeting all seven gives the best quality and costs the most memory; attention-only is the cheapest useful subset.
`bias`	none	`none`, `all`, `lora_only`	Which bias terms are trainable. 'none' keeps the adapter smallest and is almost always correct; 'lora_only' trains biases on adapted layers; 'all' trains every bias and prevents a clean merge back into the base.

Which matter most. Sequence length and rank dominate memory. Rank and the choice of target modules dominate quality. Epochs dominate whether the model learns the material or memorises it.

A note on `bias`. Only `"none"` merges cleanly. `"all"` trains bias terms that have no place to be folded into a weight matrix, so a model trained that way cannot be merged into a plain model.

Supervised fine-tuning

Table 9 — Supervised fine-tuning parameters

Parameter	Default	Accepts	What it does
`num_train_epochs`	15	1 to 1000	How many times the trainer walks the whole dataset. Small instruction sets need many passes to take effect; large ones overfit quickly. If answers start being reproduced verbatim, reduce this first.
`per_device_train_batch_size`	1	1 to 64	Examples processed together in one forward and backward pass. This is the single largest driver of VRAM use. Leave at 1 on consumer GPUs and raise the accumulation steps instead.
`gradient_accumulation_steps`	2	1 to 256	Number of batches accumulated before weights are updated. The effective batch size is this multiplied by the batch size per device, so it buys the stability of a larger batch at almost no extra VRAM cost.
`learning_rate`	0.0005	1e-06 to 0.01	Step size for the optimiser. LoRA tolerates rates one to two orders of magnitude above full fine-tuning because only the adapters move. 1e-4 to 5e-4 is the usual band; lower it if the loss oscillates.
`warmup_steps`	5	0 to 10000	Steps spent ramping the learning rate up from zero, which avoids a destructive first update on freshly initialised adapters. A handful is enough for short runs.
`max_length`	2048	128 to 131072	Token budget per training example; longer conversations are truncated. Memory grows with this value, so it is the most effective dial to turn when a run runs out of VRAM. Set it just above your longest example.

Parameter	Default	Accepts	What it does
`bf16`	True	true / false	Run the forward and backward passes in bfloat16. Halves activation memory against float32 and, unlike float16, needs no loss scaling. Requires an Ampere or newer GPU.
`gradient_checkpointing`	True	true / false	Discard intermediate activations and recompute them during the backward pass. Cuts activation memory substantially at roughly 20-30% slower training. Usually the difference between fitting and not fitting.

Effective batch size is the per-device batch size multiplied by the accumulation steps. Accumulation buys the stability of a larger batch at almost no memory cost, which is why the batch size stays at 1 on consumer GPUs and the accumulation steps go up instead.

Learning rate. LoRA tolerates rates one to two orders of magnitude above full fine-tuning, because only the small adapter matrices move. 1e-4 to 5e-4 is the usual band. If the loss oscillates rather than descending, lower it.

Epochs. The default of 15 suits a small instruction set of a few dozen examples, where a handful of passes would leave no trace. For thousands of examples it is far too many: if the model starts reproducing training answers verbatim, this is the first thing to reduce.

Gradient checkpointing discards intermediate activations and recomputes them during the backward pass — substantially less memory for roughly 20–30% slower training. On a consumer GPU it is usually the difference between fitting and not.

Quantization parameters

The output type is the only choice. Types marked as requiring `llama-quantize` are produced by the two-pass route and are unavailable when that binary is not installed.

Table 10 — GGUF output types and the tool each one needs

Type	Two-pass	Description
`f32`	no	No quantization. Four bytes per weight; a reference baseline, rarely worth shipping.
`f16`	no	Two bytes per weight, effectively lossless for inference. Also the staging format for k-quants.
`bf16`	no	Two bytes per weight with the exponent range of float32. Preferred over F16 for models trained in bfloat16.
`q8_0`	no	Roughly one byte per weight and near-indistinguishable from F16. The default: the best quality-to-size trade for most users.
`tq1_0`	no	Extreme compression for ternary-trained models. Produces unusable output on ordinary models.
`tq2_0`	no	As TQ1_0, slightly larger and more accurate. Only for models trained ternary.
`q2_K`	yes	Smallest usable k-quant. Noticeable quality loss; for when memory is the binding constraint.
`q3_K_S`	yes	Aggressive 3-bit packing with uniform block sizes.
`q3_K_M`	yes	3-bit with more precision spent on attention weights. Better than Q3_K_S at a small size cost.
`q3_K_L`	yes	3-bit with the highest-impact tensors kept larger. Approaches Q4_K_S quality.
`q4_K_S`	yes	Compact 4-bit k-quant. A good floor for quality-sensitive work.
`q4_K_M`	yes	The community default for local inference: about half the size of Q8_0 with modest quality loss.
`q5_K_S`	yes	5-bit k-quant; quality close to Q8_0 at a meaningfully smaller size.
`q5_K_M`	yes	5-bit with extra precision on sensitive tensors. Very close to lossless in practice.
`q6_K`	yes	Practically indistinguishable from Q8_0 while noticeably smaller.

The quantization block size and the per-tensor precision assignments are llama.cpp's own and are not exposed: they are properties of each named type rather than independent settings.